SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS: **Total Marks: 25**

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

A.**REST vs SOAP for a Mobile and Web Application**

A company developing a web service for both **mobile and web applications** must choose an architecture that provides good **performance, scalability, security, and interoperability**. The two common approaches are **REST (Representational State Transfer)** and **SOAP (Simple Object Access Protocol)**.

For this scenario, **REST is the more appropriate choice** because mobile and web applications generally benefit from lightweight communication, fast responses, and easy scalability.

1. Comparison of REST and SOAP

Feature	REST	SOAP
Architecture	Architectural style	Messaging protocol
Data format	Usually JSON; can also use XML	Primarily XML
Message size	Lightweight	Larger and more verbose
Performance	Generally faster	Can have additional processing overhead
Scalability	Highly scalable due to stateless design	Can be more complex to scale
Caching	Supports HTTP caching	Limited built-in HTTP caching advantages

Security	HTTPS, OAuth 2.0, JWT, API keys	WS-Security, XML encryption, signatures
Ease of use	Simple and widely supported	More complex
Mobile suitability	Very suitable	Less suitable due to XML overhead
Formal contract	OpenAPI commonly used	WSDL
Reliability	Relies mainly on HTTP and application design	Supports enterprise WS-* standards
Best suited for	Web/mobile APIs and public services	Enterprise and highly structured integrations

2. Analysis of the REST Approach

REST uses standard HTTP methods to access and manipulate resources.

For example, an application managing users and products may provide:

```
GET    /api/users/101      → Retrieve user details
POST   /api/users          → Create a new user
PUT    /api/users/101      → Update user details
DELETE /api/users/101      → Delete a user
```

REST commonly exchanges data using JSON:

```
{
  "id": 101,
  "name": "Arun Kumar",
  "email": "arun@example.com"
}
```

Advantages of REST

- Lightweight JSON messages reduce bandwidth consumption.
- Easy integration with browsers and mobile applications.
- Stateless requests make horizontal scaling easier.
- HTTP caching can improve performance.
- Large ecosystem of tools and frameworks.
- Simple endpoints and HTTP methods make APIs easier to maintain.

Limitations

- REST does not provide the same built-in enterprise messaging standards as SOAP.
- Features such as reliable delivery and advanced message-level security must often be implemented using additional mechanisms or infrastructure.

3. Analysis of the SOAP Approach

SOAP exchanges structured XML messages using a standard envelope.

Example:

```
<soap:Envelope>
   <soap:Header>
      <!-- Security information -->
   </soap:Header>

   <soap:Body>
      <getUser>
         <userId>101</userId>
      </getUser>
   </soap:Body>
</soap:Envelope>
```

Advantages of SOAP

- Strict message structure.
- Strong contract definition through WSDL.
- Standardized SOAP Fault error handling.
- Support for WS-Security.
- Suitable for complex enterprise and transactional systems.

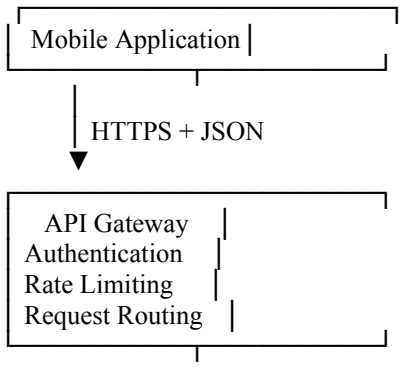
Limitations

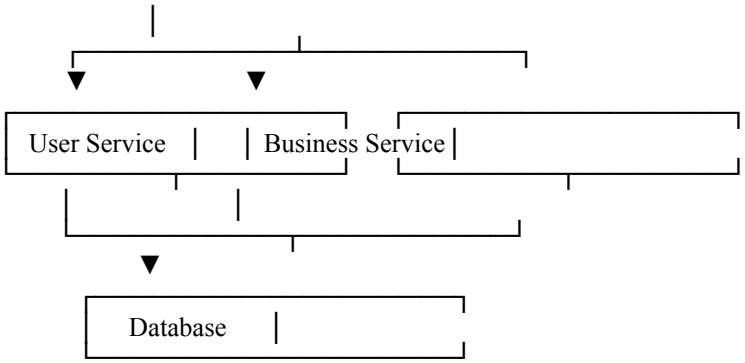
- XML messages are larger than JSON messages.
- Additional processing may increase latency.
- More complex to implement and consume.
- Less convenient for bandwidth-constrained mobile applications.

4. Selected Solution: RESTful Web Service

For a typical mobile and web application, I recommend a **RESTful API using JSON over HTTPS**.

Proposed Architecture





The web application and mobile application communicate with the same REST API.

5. Example REST API Design

Suppose the application manages customers.

Retrieve a Customer

GET /api/customers/101

Response:

```
{
  "customerId": 101,
  "name": "Arun Kumar",
  "email": "arun@example.com"
}
```

Create a Customer

POST /api/customers

Request body:

```
{
  "name": "Arun Kumar",
  "email": "arun@example.com"
}
```

Response:

HTTP/1.1 201 Created

Update a Customer

PUT /api/customers/101

Delete a Customer

DELETE /api/customers/101

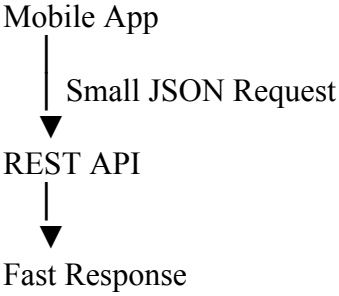
This design uses standard HTTP methods and resource-oriented URLs.

6. Justification Based on Performance

REST is preferable for performance because:

- **JSON is generally smaller and simpler to process than verbose SOAP XML.**
- Smaller requests and responses reduce bandwidth usage, which is especially important for mobile networks.
- REST can use HTTP features such as **compression and caching**.
- Fewer layers of XML processing can reduce client and server overhead.

For example:

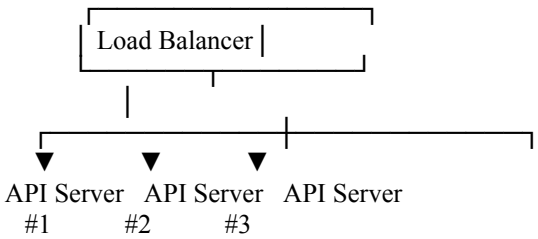


Therefore, REST is well suited for applications where users expect responsive interfaces.

7. Justification Based on Scalability

REST is naturally well suited to scalable architectures because it is typically **stateless**.

Each request contains the information needed for processing, and the server does not need to maintain client session state between requests.



Because requests can be routed to multiple instances, additional servers can be added as demand increases.

Caching frequently requested data can further reduce database load.

8. Justification Based on Security

Although SOAP provides strong built-in enterprise security standards such as WS-Security, REST can provide robust security for mobile and web applications when properly designed.

The proposed REST API should use:

HTTPS/TLS

Encrypts communication between the client and server.

OAuth 2.0 or OpenID Connect

Provides standards-based authentication and authorization flows, particularly suitable for mobile and web clients.

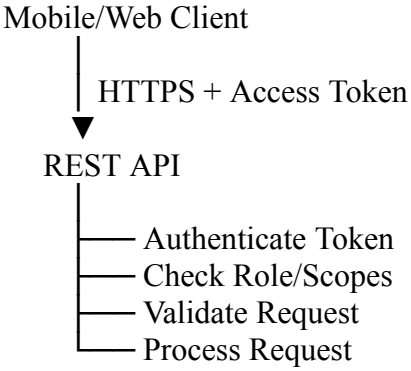
Short-Lived Access Tokens

Limits the impact of a compromised token.

Role-Based or Scope-Based Authorization

Ensures users can access only permitted resources.

For example:



Additional protections should include:

- Input validation
- Rate limiting
- Secure password storage
- Logging and monitoring
- Protection against common web attacks
- Proper CORS configuration for browser clients

9. When SOAP Would Be a Better Choice

SOAP could be preferred if the company specifically requires:

- Strict WSDL-based contracts
- WS-Security at the message level
- Complex enterprise transactions
- Legacy enterprise system integration
- Formal WS-* standards for reliable messaging

For example, a banking or large enterprise integration system involving multiple legacy systems may select SOAP for certain internal services.

However, these requirements are not generally necessary for a typical mobile and web application API.

Conclusion

Both REST and SOAP can be used to develop web services, but they are suitable for different requirements. **SOAP** is valuable for highly structured enterprise integrations requiring formal contracts and WS-* standards. However, for a company building a service primarily for **mobile and web applications**, **REST is the recommended approach**.

The proposed solution is a **stateless RESTful API using JSON over HTTPS**, protected with standards-based authentication and authorization and deployed behind scalable infrastructure. REST provides **better performance due to lightweight messages, easier scalability through stateless communication, and strong security when implemented with TLS, secure authentication, authorization, and proper API controls**.

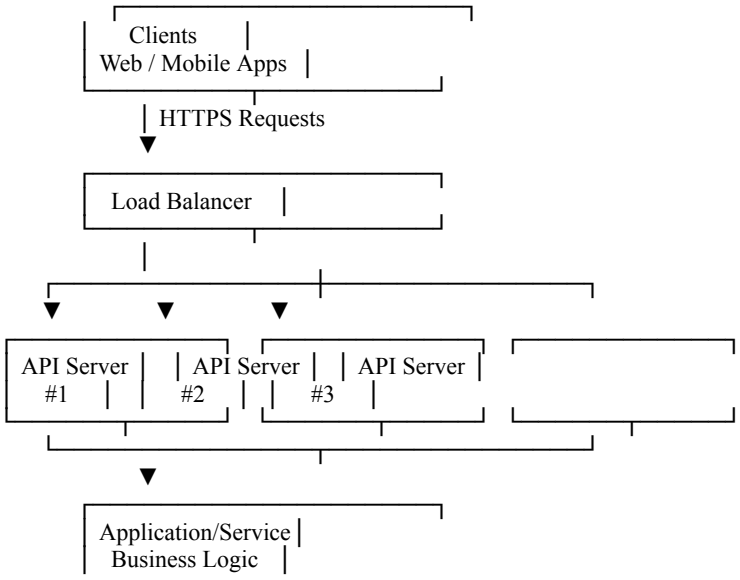
2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

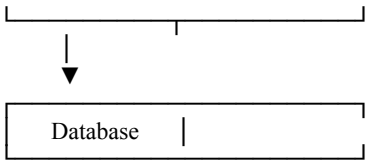
A. **Scalable Service Architecture for an Online Platform**

An online platform expecting a large number of simultaneous users needs an architecture that can distribute traffic, avoid single points of failure, and scale individual components independently.

1. Basic Scalable Architecture

A basic architecture can be designed as follows:





Components

- **Clients:** Web browsers and mobile applications send requests.
- **Load Balancer:** Distributes incoming traffic across multiple API servers.
- **API Servers:** Handle authentication, validation, and API requests.
- **Application Services:** Execute business logic.
- **Database:** Stores users, transactions, products, and other application data.

Working

1. A client sends an HTTPS request.
2. The load balancer receives the request.
3. The request is routed to an available API server.
4. The API server processes the request and executes business logic.
5. Required data is read from or written to the database.
6. The response is returned to the client.

This architecture is better than a single-server design because additional API servers can be added when traffic increases.

2. Limitations of the Basic Architecture

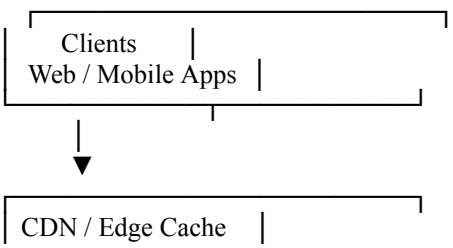
Although the architecture can scale the API layer, problems may still occur:

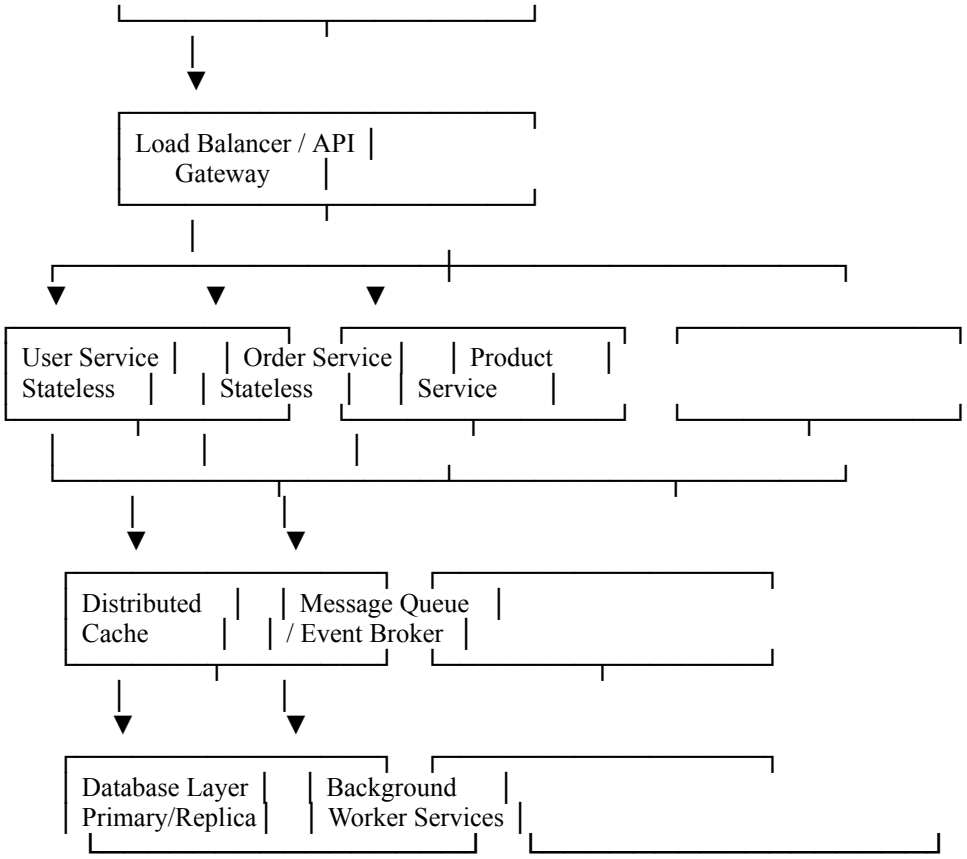
- The **single database** can become a bottleneck.
- Repeated database queries may increase response time.
- Long-running tasks can occupy API server resources.
- A failure in a shared service may affect many users.
- Scaling the entire application together may waste resources.

Therefore, a more scalable alternative can be introduced.

3. Alternative Design: Distributed and Horizontally Scalable Architecture

A better design is to use an **API gateway, stateless services, caching, asynchronous processing, and a scalable database layer**.





4. Improvements in the Alternative Design

A. API Gateway

The API Gateway acts as a single entry point for clients.

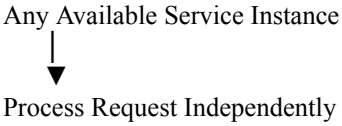
Its responsibilities can include:

- Request routing
- Authentication and authorization
- Rate limiting
- Request logging
- API versioning
- Load distribution

Clients do not need to communicate directly with every internal service.

B. Stateless Application Services

Services should be designed to be **stateless** wherever possible.



Since no user session is stored only on one server, a request can be handled by any available instance.

This allows easy **horizontal scaling**:

Normal Load: [Server 1] [Server 2]

High Load: [Server 1] [Server 2] [Server 3] [Server 4]

An orchestration platform can automatically add or remove instances according to CPU usage, request rate, or other metrics.

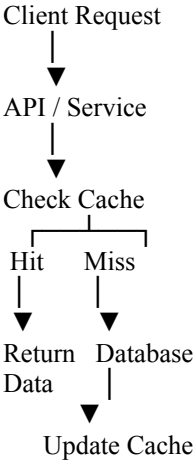
C. Distributed Cache

Frequently requested information should be stored temporarily in a cache.

Examples include:

- Product information
- User preferences
- Frequently accessed public content
- Session-related data, if required

The flow becomes:



This reduces the number of database requests and improves response time.

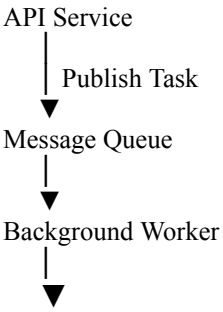
D. Asynchronous Processing Using a Message Queue

Long-running tasks should not block the main user request.

Examples:

- Sending emails
- Generating reports
- Processing uploaded files
- Sending notifications

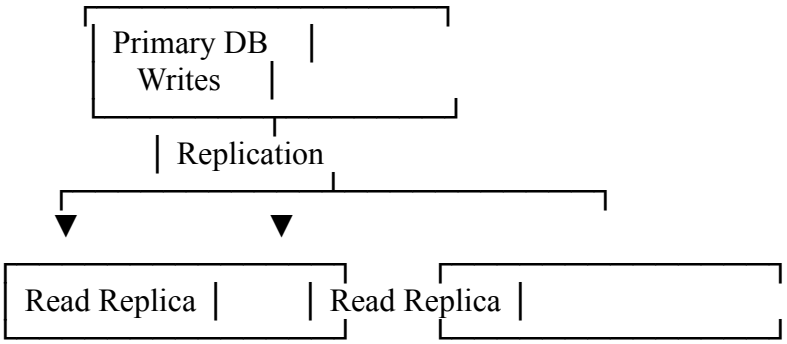
Instead:



This improves responsiveness because the client does not need to wait for the entire background operation to finish.

E. Database Scaling

The database can be improved using a **primary-replica architecture**.



- Write operations go to the primary database.
- Read-heavy traffic can be distributed across replicas.

For extremely large datasets, techniques such as **partitioning or sharding** may also be used.

5. Comparison of the Two Designs

Aspect	Basic Architecture	Alternative Architecture
Client entry	Load balancer	CDN + API Gateway
API scaling	Multiple servers	Independently scalable services
Application design	Centralized business layer	Distributed services
Caching	Optional/limited	Distributed cache

Background tasks	Often synchronous	Asynchronous queue
Database	Single database	Primary + replicas
Fault isolation	Lower	Higher
Scalability	Moderate	High
Complexity	Lower	Higher

6. Justification for the Alternative Design

The alternative architecture is more suitable for a platform expecting a very large number of simultaneous users because it removes several potential bottlenecks.

Improved Performance

CDNs and caches reduce repeated requests to application servers and databases.

Horizontal Scalability

Stateless API and service instances can be increased independently as traffic grows.

Database Scalability

Read replicas distribute read operations and reduce pressure on the primary database.

Better Responsiveness

Message queues move slow, non-critical tasks to background workers.

Higher Reliability

Multiple instances reduce dependence on a single server. If one service instance fails, traffic can be redirected to other healthy instances.

Independent Scaling

For example, if product searches become highly popular, only the Product Service can be scaled instead of scaling the entire system.

Conclusion

A basic architecture using **clients → load balancer → multiple API servers → application service → database** provides initial horizontal scalability.

For a high-traffic online platform, the recommended alternative is a **distributed architecture with a CDN, API gateway, stateless services, distributed cache, message queue, background workers, and a scalable database layer**. This approach improves

performance, scalability, responsiveness, fault tolerance, and reliability, making it more suitable for handling a large and growing number of simultaneous users.

3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.

A.Secure Communication Protocol for a Banking Application

A banking application handles highly sensitive information such as login credentials, account details, balances, and financial transactions. Therefore, communication between the client and server must provide **confidentiality, integrity, authentication, reliability, and good performance**.

The most suitable choice for typical banking client-server communication is **HTTPS using TLS**.

1. Analysis of Available Protocols

A. HTTP

HTTP (Hypertext Transfer Protocol) is commonly used for web communication.

Client ——— HTTP ———> Server

However, standard HTTP does not encrypt transmitted data.

Advantages:

- Simple and fast
- Widely supported

Disadvantages:

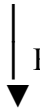
- No confidentiality
- Data may be intercepted or modified
- Not suitable for banking transactions

Conclusion: HTTP alone should not be used.

B. HTTPS

HTTPS (HTTP Secure) uses HTTP over **TLS (Transport Layer Security)**.

Client



Encrypted HTTPS/TLS Connection

Banking Server

TLS provides:

- **Confidentiality** through encryption
- **Integrity** through authenticated protection against undetected modification
- **Server authentication** using digital certificates
- Optional client authentication where required

Conclusion: HTTPS/TLS is highly suitable for secure banking applications.

C. FTP

FTP is designed primarily for file transfer and is not appropriate for normal banking application communication.

Standard FTP may expose credentials and transferred data unless additional security mechanisms are used.

Conclusion: Not suitable.

D. SSH

SSH provides encrypted remote access and secure command execution.

It is useful for:

- Secure server administration
- Secure remote management
- Secure tunneling

However, it is not normally the protocol used between banking mobile/web clients and application APIs.

Conclusion: Useful for administration, but not the primary client-server protocol.

E. SOAP with WS-Security

SOAP can provide strong enterprise-level security through **WS-Security**, including:

- Message-level encryption
- Digital signatures
- Security tokens
- Authentication information

SOAP is useful when banking systems need formal contracts, strict interoperability, and message-level security across multiple intermediaries.

However, SOAP messages are XML-based and can be more verbose, resulting in greater processing and bandwidth overhead.

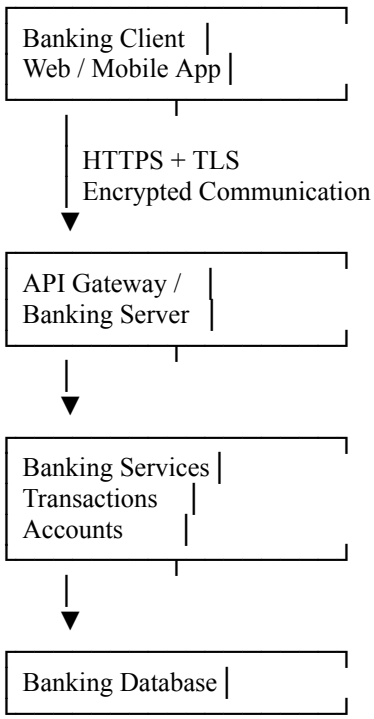
Conclusion: A strong option for some internal or legacy banking integrations, but not necessarily the best default protocol for modern mobile and web client communication.

2. Selected Protocol: HTTPS with TLS

The recommended protocol is:

> **HTTPS using modern TLS, typically over HTTP/2 or HTTP/3 where supported.**

The communication model is:



3. Justification Based on Security

Security is the most important requirement for banking communication.

A. Confidentiality

TLS encrypts data exchanged between the client and server.

Without encryption:
Username + Password → Readable to an attacker

With TLS:
Encrypted data → Cannot be meaningfully read by an interceptor

This protects:

- Login credentials
- Account information
- Transaction details
- Personal data

- Authentication tokens

B. Server Authentication

The server presents a digital certificate during the TLS connection setup. The client validates the server's identity through the certificate validation process.

This helps prevent connections to an unauthorized server.

For sensitive banking applications, additional controls can include certificate pinning where operationally appropriate.

C. Message Integrity

TLS provides integrity protection for data in transit. If protected traffic is modified during transmission, the connection's authentication mechanisms detect the alteration.

This helps protect transaction requests from undetected tampering.

D. Additional Authentication and Authorization

HTTPS should be combined with application-level security, such as:

HTTPS/TLS
  +
Strong User Authentication
  +
Session/Token Protection
  +
Authorization Controls
  +
Transaction Validation

Possible mechanisms include:

- Multi-factor authentication
- Short-lived access tokens
- Secure session management
- Role-based or attribute-based authorization
- Transaction confirmation for high-risk operations

HTTPS protects the communication channel, while these mechanisms control **who is allowed to perform specific actions**.

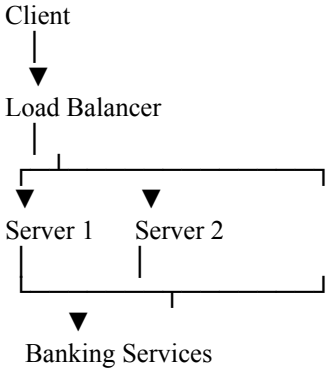
4. Justification Based on Reliability

HTTPS is built on reliable application communication mechanisms and benefits from mature HTTP infrastructure.

Reliability can be improved using:

- Load balancing
- Redundant servers
- Health checks
- Failover mechanisms
- Timeouts
- Careful retry policies
- Idempotency keys for transaction operations

Example:



If one server fails, requests can be routed to another healthy server.

For financial transactions, retries must be designed carefully. Simply repeating a failed transfer request could result in duplicate processing. Therefore, transaction IDs and idempotency controls should be used.

5. Justification Based on Performance

HTTPS adds encryption and TLS processing overhead compared with plain HTTP, but modern systems handle this efficiently.

Performance can be improved through:

HTTP/2 or HTTP/3

These can improve connection and request efficiency compared with older HTTP usage patterns.

Connection Reuse

Persistent connections reduce the cost of repeatedly establishing secure connections.

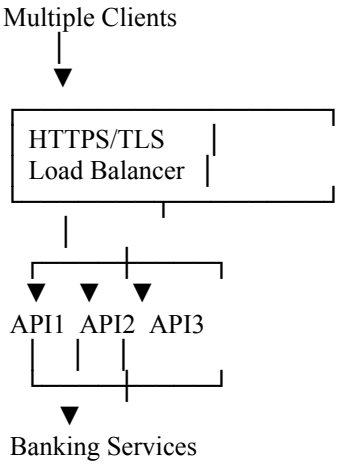
Load Balancing

Requests can be distributed across multiple servers.

Scalable Infrastructure

Multiple application instances can process requests simultaneously.

The architecture can therefore provide both strong security and good performance:



6. Comparison Summary

Protocol/Approach	Security	Reliability	Performance	Suitability for Banking Client APIs
HTTP	Low	Moderate	High	Not suitable
HTTPS/TLS	Very high	High	High	**Highly suitable**
FTP	Low without added security	Moderate	Moderate	Not suitable
SSH	High	High	Moderate	Mainly administration
SOAP + WS-Security	Very high	Very high	Moderate	Suitable for enterprise/internal integrations

Conclusion

For a modern banking application, the recommended communication protocol is **HTTPS using modern TLS**. It provides strong protection for data in transit through **encryption, integrity protection, and server authentication**, while remaining widely supported and efficient for web and mobile clients.

Reliability should be strengthened with **redundant infrastructure, failover, timeouts, carefully controlled retries, and idempotent transaction processing**. HTTPS should also be combined with **strong user authentication, authorization, secure session/token handling, and transaction-level validation**.

Therefore, **HTTPS/TLS provides the best overall balance of security, reliability, performance, and compatibility for client-server communication in a banking application**, while SOAP with WS-Security may still be appropriate for specialized internal banking integrations requiring message-level enterprise security.

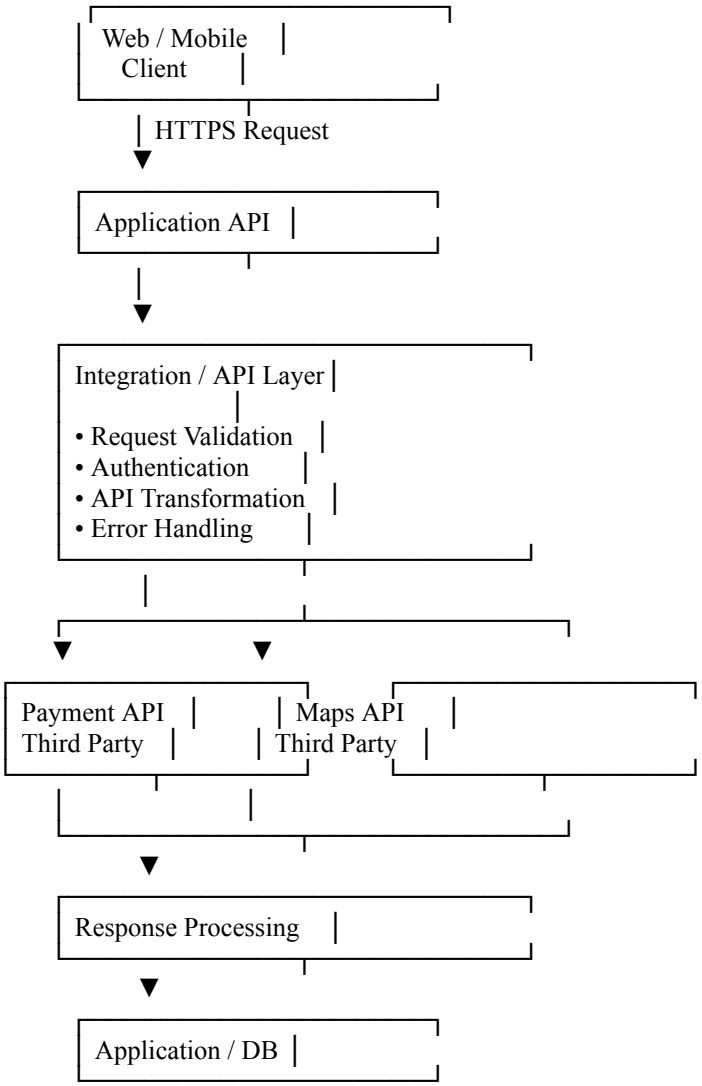
4.An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.

A.API Integration Strategy for Third-Party Services

An application that integrates with third-party services such as **payment gateways** or **map services** needs a structured approach for sending requests, authenticating securely, processing responses, and handling failures.

A good design should avoid connecting the application directly to every external API from the client. Instead, third-party integrations should generally be managed through a dedicated backend integration layer.

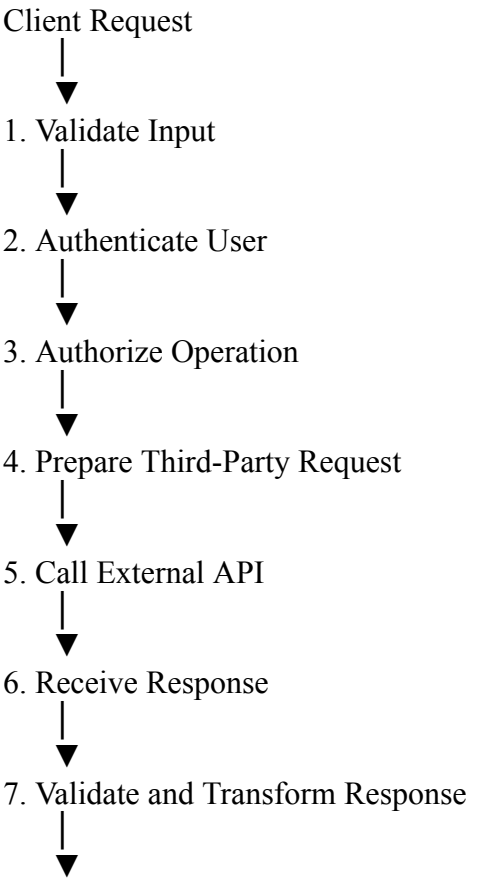
1. Proposed API Integration Architecture



The **integration layer** acts as an intermediary between the application and external services.

2. Request Handling Strategy

The application should process a request in the following sequence:



Step 1: Validate the Request

Before contacting a third-party API, validate:

- Required fields
- Data types
- Input formats
- Amount values for payments
- Location or address formats for map requests

Example:

```
{
  "amount": 1500,
  "currency": "INR",
  "orderId": "ORD-1001"
}
```

Invalid requests should be rejected immediately instead of unnecessarily calling the third-party service.

3. Authentication Strategy

Different third-party APIs use different authentication mechanisms.

A. API Keys

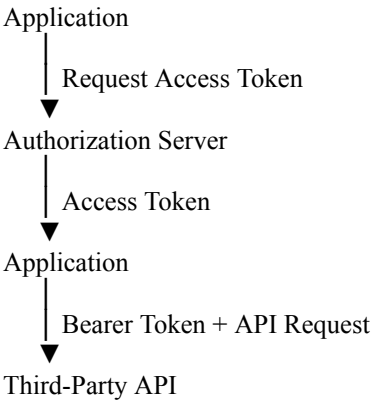
Some map or information services use an API key.

GET /maps?location=Chennai
Authorization: API-Key <API_KEY>

The API key should be stored securely on the server or in a secure secret-management system, not exposed unnecessarily in client-side code.

B. OAuth 2.0

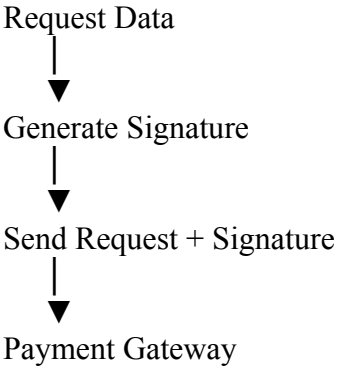
OAuth 2.0 may be used when an application requires delegated authorization or access tokens.



Tokens should be handled securely and refreshed before expiry when refresh tokens or another renewal mechanism are supported.

C. Signed Requests

Some payment services require requests to include a cryptographic signature.

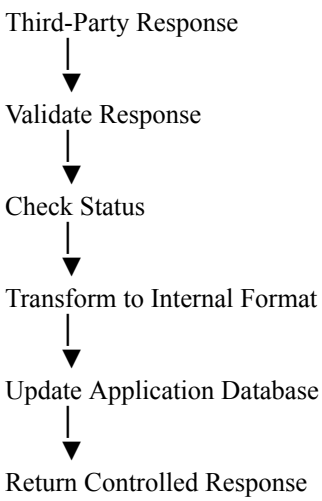


This helps the provider verify that the request has not been altered and originates from an authorized integration.

4. Response Processing

The third-party response should not always be passed directly to the client.

Instead:



For example, different payment providers may return different response formats.

Provider Response

```
{
  "transaction_id": "TX-789",
  "payment_status": "completed",
  "provider_message": "Payment successful"
}
```

The integration layer can convert it into a standard internal response:

```
{
  "transactionId": "TX-789",
  "status": "SUCCESS",
  "message": "Payment completed successfully"
}
```

This reduces dependency between the rest of the application and a specific provider's API format.

5. Error Handling

The application should distinguish between different categories of errors.

Error	Example	Action

Client error	Invalid payment amount	Return validation error
Authentication error	Invalid API key	Refresh/correct credentials and alert administrators
Network error	Connection failure	Retry when safe
Timeout	Provider is slow	Use timeout and controlled retry
Server error	Provider returns 500	Retry with backoff if appropriate
Business error	Payment declined	Return meaningful result; usually do not blindly retry

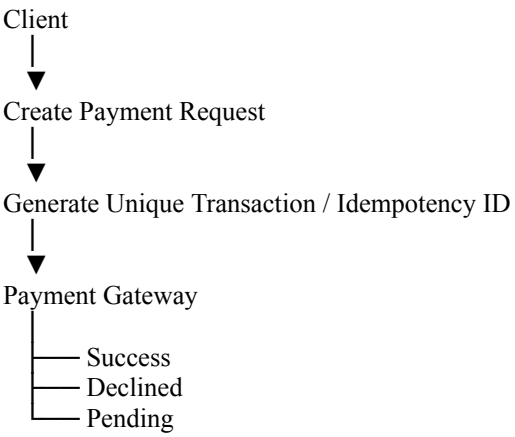
A standard internal error format can be used:

```
{
  "status": "ERROR",
  "code": "PAYMENT_TIMEOUT",
  "message": "The payment service did not respond in time"
}
```

Detailed provider errors should be logged securely, while sensitive information should not be exposed to end users.

6. Payment Integration Considerations

Payment operations require additional reliability controls.

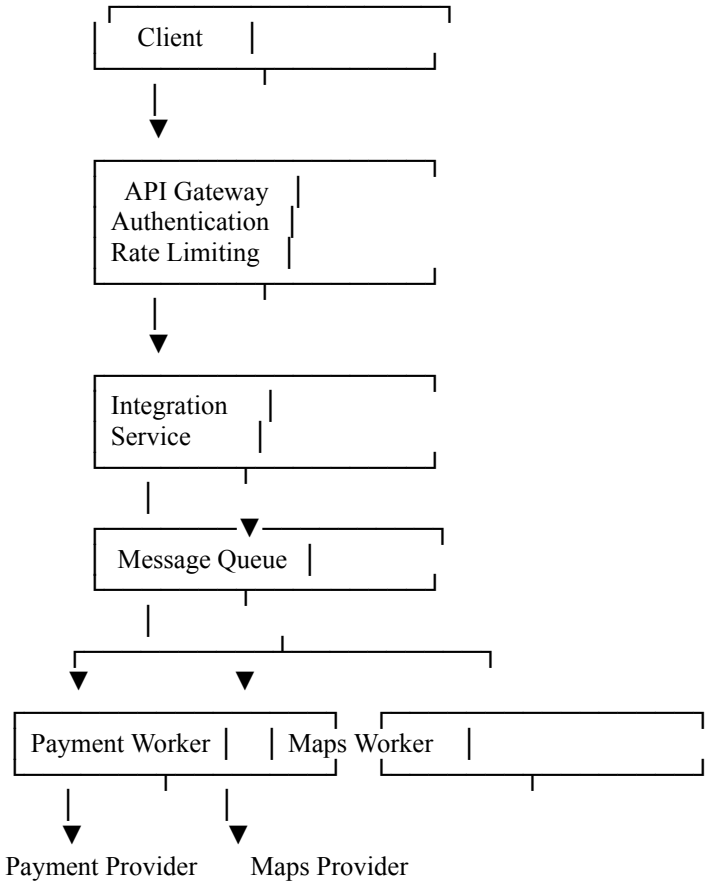


The application should use a unique transaction or idempotency identifier where supported. This prevents a retry after a timeout from accidentally creating or charging the same payment twice.

The final payment status should be verified through the provider's trusted confirmation mechanism, such as a signed webhook or status query.

7. Alternative Approach: API Gateway and Asynchronous Integration

For greater efficiency and reliability, an alternative design can use an **API Gateway combined with a message queue and asynchronous workers**.



Advantages

- Long-running operations do not block client requests.
- Requests can be queued during temporary provider outages.
- Workers can be scaled independently.
- Controlled retries can be implemented.
- Failed messages can be moved to a **dead-letter queue** for investigation.
- Different third-party integrations can be isolated from one another.

For **real-time map lookups**, synchronous API calls may still be appropriate because the user usually expects an immediate response. For operations that can tolerate delayed completion, asynchronous processing is more resilient.

8. Additional Reliability Improvements

The following techniques can improve the integration further:

Circuit Breaker

Temporarily stops repeated requests to an unhealthy provider, preventing cascading failures.

Retry with Exponential Backoff

Retries transient failures after increasing delays. Retries should only be used when the operation is safe to repeat.

Timeouts

Prevents application threads or connections from waiting indefinitely for an external service.

Webhooks

Allows a provider to notify the application when an asynchronous operation is completed.

Monitoring and Logging

Track:

- Request ID
- Provider name
- Response time
- Success/failure rate
- Error type

Sensitive credentials, payment data, and authorization tokens must be masked from logs.

Conclusion

A recommended API integration strategy is to use a dedicated **backend integration layer** that validates requests, securely manages authentication, sends requests to third-party APIs, validates and transforms responses, and handles errors consistently.

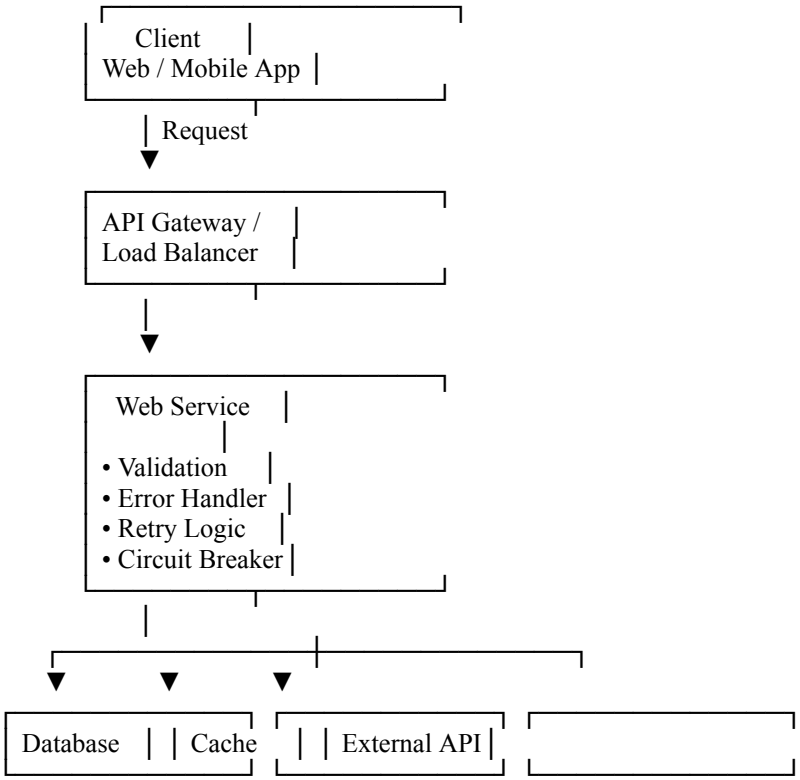
For higher reliability, the architecture can be improved with an **API Gateway, asynchronous message queues, independently scalable workers, timeouts, circuit breakers, controlled retries, and dead-letter queues**. This design reduces coupling with third-party providers and makes the application more **secure, efficient, scalable, and resilient to external service failures**.

5.A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

A.Error Handling and Fault Tolerance Mechanism for a Web Service

A web service that frequently encounters errors should use a structured mechanism to **detect, classify, handle, recover from, and monitor failures**. The main goals are to maintain system availability, prevent cascading failures, protect data integrity, and provide users with clear responses

1. Proposed Error Handling Architecture



The service should handle errors at each stage rather than allowing an unhandled exception to reach the user.

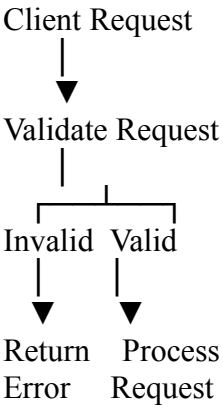
2. Error Classification

Errors should first be classified because different failures require different responses.

Error Type	Example	Handling Strategy
Client error	Invalid input	Return validation error
Authentication error	Invalid token	Return authentication error
Authorization error	Access denied	Return permission error
Not found	Resource does not exist	Return 404-style response
Server error	Application exception	Log and return safe error
Timeout	Slow external service	Timeout and controlled retry
Database error	DB temporarily unavailable	Failover or retry
External API failure	Third-party service unavailable	Circuit breaker/fallback

3. Request Validation

The first level of protection is validating requests before processing them.



Validation should check:

- Required parameters
- Data types
- Data ranges
- Request format
- Authentication credentials
- Business rules

Example response:

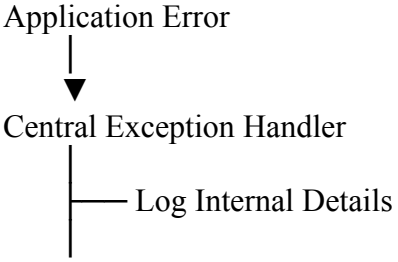
```
{
  "status": "ERROR",
  "code": "INVALID_REQUEST",
  "message": "The required customer ID is missing."
}
```

Early validation reduces unnecessary processing and prevents invalid requests from causing server errors.

4. Centralized Exception Handling

The web service should use a centralized error-handling component.

Instead of returning raw system exceptions, errors should be converted into a standard format.



└── Return Safe Response

Example:

```
{
  "status": "ERROR",
  "code": "INTERNAL_ERROR",
  "message": "The request could not be completed.",
  "requestId": "REQ-78451"
}
```

The detailed stack trace should be stored internally and not exposed to the client.

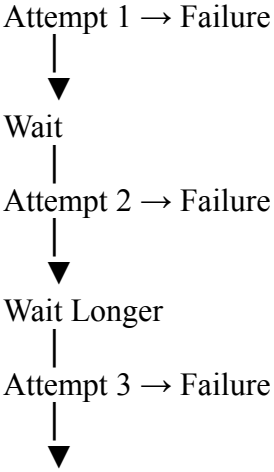
5. Retry Mechanism

Temporary failures can sometimes be resolved by retrying the operation.

Examples include:

- Temporary network failures
- Short database connection failures
- Temporary external API errors

A controlled retry policy can be:



Exponential backoff should be used rather than immediately repeating requests.

For example:

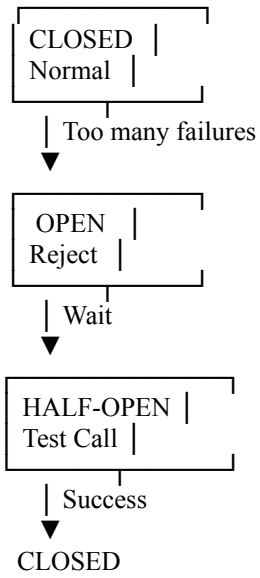
Retry 1 → wait 1 second
Retry 2 → wait 2 seconds
Retry 3 → wait 4 seconds

Retries should only be used for operations that are safe to repeat. For payment or transaction operations, use an **idempotency key** or unique request ID to prevent duplicate processing.

6. Circuit Breaker

If an external service repeatedly fails, continuously sending requests can make the problem worse.

A circuit breaker has three common states:



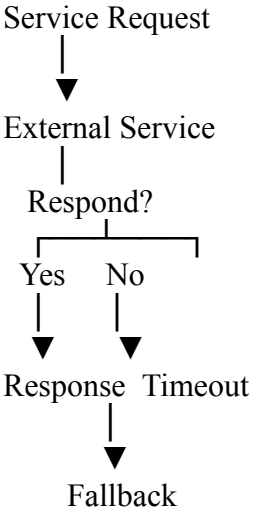
When the circuit is open:

- Requests are not repeatedly sent to the failing service.
- The application can return a fallback response.
- Periodic test requests check whether the service has recovered.

This prevents cascading failures.

7. Timeout and Fallback Mechanism

Every external or database call should have appropriate timeout limits.



A fallback may include:

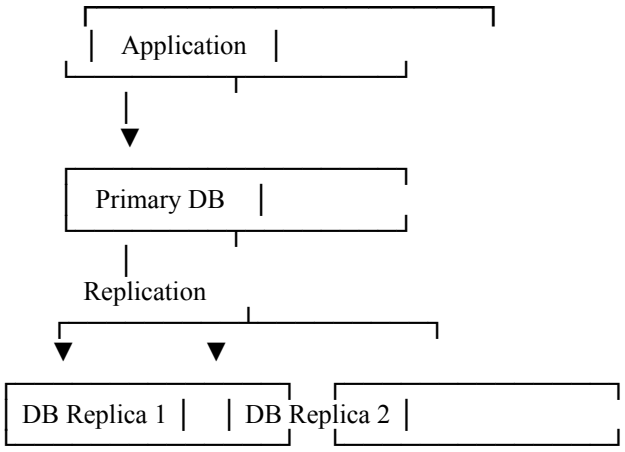
- Cached data
- A default response
- A temporary "service unavailable" response
- Queueing the request for later processing

For example, if a recommendation service is unavailable, the application may display cached recommendations instead.

8. Database Fault Tolerance

The database is often a critical dependency.

A fault-tolerant design can use:



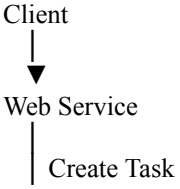
Possible strategies include:

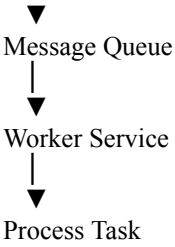
- Database replication
- Automatic failover
- Connection pooling
- Backups and recovery procedures
- Read replicas for read-heavy workloads

Critical write operations should use appropriate consistency and transaction guarantees rather than blindly retrying.

9. Alternative Strategy: Asynchronous Message Queue

For operations that do not require an immediate result, use asynchronous processing.





If a worker fails, the message can remain available for another worker to process.

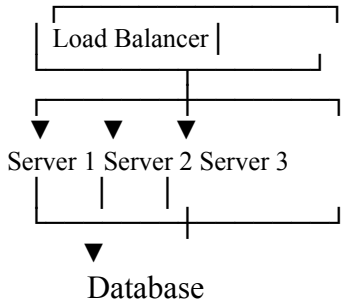
Advantages include:

- Reduced load on the main API
- Better handling of traffic spikes
- Retry support
- Independent worker scaling
- Improved resilience

A **dead-letter queue** can store messages that repeatedly fail for later investigation.

10. Alternative Strategy: Redundancy and Load Balancing

Another approach is to deploy multiple service instances.



If one server fails, the load balancer routes requests to healthy servers.

Health checks can automatically remove failed instances from service.

This approach improves:

- Availability
- Fault tolerance
- Scalability

11. Monitoring and Logging

The system should continuously monitor errors and performance.

Useful information includes:

- Request ID

- Error code
- Service name
- Timestamp
- Response time
- Retry count
- Dependency status

Example:

Request ID: REQ-78451
Service: OrderService
Dependency: PaymentAPI
Error: Timeout
Retry Count: 2
Final Status: Failed

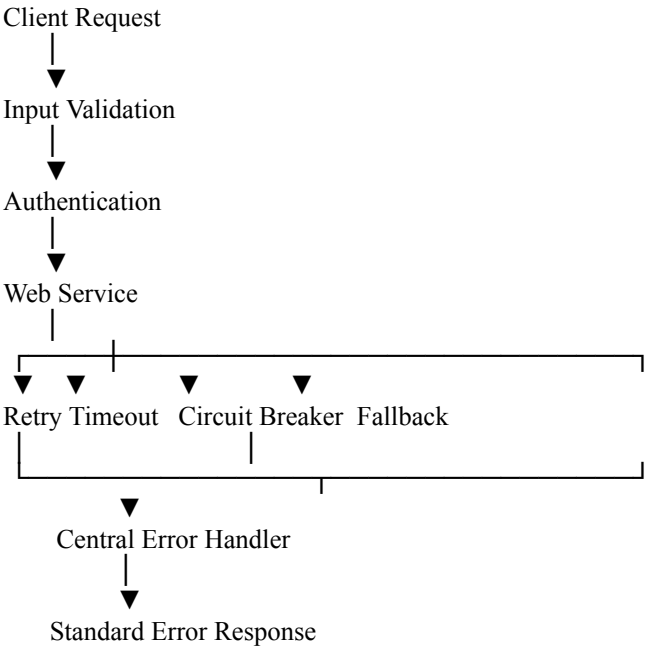
Sensitive information such as passwords, tokens, and personal data should not be written to logs.

Monitoring systems can trigger alerts when:

- Error rates increase
- Response times become high
- A dependency fails
- Server resources are exhausted

12. Recommended Combined Solution

A reliable web service should combine several techniques:



The recommended strategy is:

1. Validate requests early.
2. Use centralized exception handling.
3. Apply timeouts to all remote dependencies.
4. Retry only transient and safe-to-repeat operations.
5. Use exponential backoff and idempotency controls.
6. Use circuit breakers for unstable dependencies.
7. Provide fallbacks or cached responses where appropriate.
8. Use message queues for asynchronous workloads.
9. Deploy redundant service instances behind a load balancer.
10. Monitor errors and trace requests using correlation IDs.

Conclusion

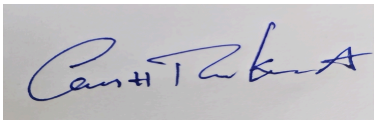
A robust error-handling and fault-tolerance mechanism should not rely on a single solution. The best approach combines **validation, centralized error handling, controlled retries, timeouts, circuit breakers, fallbacks, redundancy, database failover, asynchronous queues, and continuous monitoring**.

This layered design improves system reliability by isolating failures, preventing cascading errors, recovering from temporary problems, and providing clear responses to users. As a result, the application remains more **available, stable, and responsive**, ensuring a smoother user experience even when individual components or external services fail.

Code of Conduct:

***I* Abburi Ganesh Ram Kamal 19231118** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect